Reviewer Pack — Minimal Order of Non-Crossing Partitions and Frege Proof Dep…

Entry 843dea5d1c58 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 00:02:48 UTC

Minimal Order of Non-Crossing Partitions and Frege Proof Depth

Entry ID: `843dea5d1c58` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 00:02:48 UTC

1. Conjecture

Field A (mathematical branch): Combinatorics (Non-Crossing Partitions) **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

> For every satisfiable Boolean formula in CNF with n variables, the minimal order of a non-crossing partition of the associated poset is linearly correlated with its Frege proof depth, such that O(n) ≤ f(φ) ≤ Θ(n^2), where f(φ) represents the minimal order of a non-crossing partition and φ is the Boolean formula.

Rationale (proposer's reasoning):

> Non-crossing partitions provide a combinatorial structure that can be used to represent the search space of the Frege proof system, potentially revealing insights into its complexity. The use of non-crossing partitions in this context has not been widely explored in complexity theory, offering a novel perspective on the study of Frege proofs.

Taxonomy category: `COMBINATORICS_×_BOOLEAN_SATISFIABILITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `6a4d9e62be812ff8`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson's correlation coefficient (r) from 30 random seeds is within the range [0.8, 1], and the mean of f(φ) values across all seeds is between O(n) and Θ(n^2).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"non-crossing partitions" AND "Frege proof complexity"` - `"CNF Boolean formulas" AND minimal order AND non-crossing partitions"` - `"Frege proof depth" IN BOOLEAN LOGIC AND related TO combinatorics`

Top relevant hits considered: - [http://arxiv.org/abs/2103.07344v2] Search of fractal space-filling curves with minimal dilation - [http://arxiv.org/abs/2603.10944v1] Simple minimally unsatisfiable subsets of 2-CNFs - [http://arxiv.org/abs/2603.07606v1] TT-Sparse: Learning Sparse Rule Models with Differentiable Truth Tables - [http://arxiv.org/abs/1907.03265v4] A Neutral Temporal Deontic STIT Logic - [http://arxiv.org/abs/2312.16035v1] On three-valued presentations of classical logic - [http://arxiv.org/abs/2011.03064v1] The logic of contextuality - [s2:10.1109/ACSSC.2001.987677] Asymptotic eigenvalue moments for linear multiuser detection - [s2:10.24033/BSMF.2220] Complexity of sequences defined by billiard in the cube

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

# Helper functions for matrix operations and graph generation

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, m):
            factor = -A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] += factor * A[i][k]
    return A

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
```

```
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1) ** j * A[0][j] * determinant(submatrix)
    return det

def is_independent_set(S, poset):
    for i in S:
        for j in S:
            if i != j and poset[i][j] == 1:
                return False
    return True

def non_crossing_partition(poset, n):
    independent_sets = []
    for r in range(1, n+1):
        for subset in itertools.combinations(range(n), r):
            if is_independent_set(subset, poset):
                independent_sets.append(subset)
    partition = []
    while len(independent_sets) > 0:
        max_size = -1
        max_index = -1
        for i, s in enumerate(independent_sets):
            if len(s) > max_size:
                max_size = len(s)
                max_index = i
        partition.append(independent_sets[max_index])
        independent_sets.pop(max_index)
    return partition

def frege_proof_depth(formula):
    # Placeholder function to calculate Frege proof depth
    # This is a dummy implementation and should be replaced with actual logic
    return len(formula)

# Main trial function
def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    instances_tested = 30
    metric_values = []
    conjecture_holds = True
    counterexample = ""
```

```
    for _ in range(instances_tested):
        # Generate a random CNF formula with n variables
        num_clauses = random.randint(1, n)
        clauses = []
        for _ in range(num_clauses):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            clauses.append(clause)

        # Convert CNF to poset
        poset = [[0] * n for _ in range(n)]
        for clause in clauses:
            for lit1 in clause:
                for lit2 in clause:
                    if abs(lit1) != abs(lit2):
                        poset[abs(lit1)-1][abs(lit2)-1] = 1
                        poset[abs(lit2)-1][abs(lit1)-1] = 1

        # Compute the minimal order of a non-crossing partition
        partition = non_crossing_partition(poset, n)
        f_phi = len(partition)

        # Calculate Frege proof depth
        f_phi_depth = frege_proof_depth(clauses)

        # Store metric value
        metric_values.append(f_phi)

        # Check correlation and bounds
        if not (1 <= f_phi <= n**2):
            conjecture_holds = False
            counterexample = "f(φ) out of bounds"

    mean_value = sum(metric_values) / instances_tested
    std_value = math.sqrt(sum((x - mean_value)**2 for x in metric_values) / instances_tested)

    return {
        "metric_name": "Minimal Order of Non-Crossing Partition",
        "metric_value": mean_value,
        "instances_tested": instances_tested,
        "n_max": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
```

```
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='f(φ) out of bounds' first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means that the Pearson's correlation coefficient and mean of f(φ) values could not be calculated to verify the conjecture. | next: Re-run the test with a longer timeout or an alternative method that can produce results within a reasonable time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13815
2	preregistration	ollama_remote	glm4:latest	0	0	9719
3	novelty	ollama_remote	glm4:latest	0	0	11738
4	novelty	ollama_remote	glm4:latest	0	0	10411
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18670
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11333
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11867
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15830
9	judge	ollama_remote	glm4:latest	0	0	11910

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 115294 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/843dea5d1c58.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/843dea5d1c58.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/843dea5d1c58.tar.gz` (if generated)